MIRAI Shield ユーザーマニュアル

IIoT PQC 移行プラットフォームのための提案ガイド

目的

本マニュアルは，アップロード用データセットの準備方法，MIRAI Shield の出力の解釈方法，および IIoT 環境における構造化された耐量子暗号移行計画のためのポリシーエンジンとプロファイルレコメンダの利用方法を説明する。

著者

Borhan Rownak

Prof. Yuzo Taenaka

Prof. Youki Kadobayashi

所属

Cyber Resilience Laboratory
Division of Information Science
Nara Institute of Science and Technology
Ikoma, Nara, Japan

Version 1.0

April 9, 2026

Contents

1 はじめに **2**
1.1 概要 2
1.2 目的 2

2 プラットフォームのワークフロー **2**
2.1 ステップごとの流れ 3

3 システムモデリングとインベントリ作成 **3**
3.1 Purdue レイヤモデリング 4
3.2 Data and Impact Inventory 4
3.3 Cryptographic Inventory 4

4 アップロード用データセットの準備 **4**
4.1 重要なフォーマット規則 4
4.2 推奨される必須列 4
4.3 Unified PQC and Migration aware conduit inventory (UPMI) 属性ガイド 4

5 プラットフォームの使用方法 **6**
5.1 ステップ 1: データセットをアップロードする 6
5.2 ステップ 2: Policy Engine を実行する 6
5.3 ステップ 3: Profile Recommender を実行する 7

6 出力の解釈 **7**
6.1 Algorithm rank 7
6.2 Best algorithm 7
6.3 Confidence score 7
6.4 Migration priority 7

7 推奨レビュー手順 **8**

8 よくあるアップロード時の誤り **8**
8.1 デフォルト重みとユーザによるカスタマイズ 8

9 説明用ケーススタディ: Smart Grid UPMI 例 **11**
9.1 Example Dataset Link 11

10 結論 **11**

1. はじめに

1.1 概要

MIRAI Shield は，IIoT に特化した耐量子暗号移行フレームワークである。産業向けの Purdue レイヤベースのシステムモデリングに基づき，本フレームワークは産業資産および通信コンジットの統合暗号インベントリを作成するための段階的な方法論を提供する。この構造化インベントリに基づいて，ポリシーエンジンは各レコードを評価し，必要な PQC セキュリティカテゴリを決定し，実現可能なプロトコル更新経路を特定し，実運用上の展開制約を考慮する。その後，本フレームワークは，エビデンスに基づく移行計画を支援するために，ランク付けされた暗号推奨，信頼度スコア，および移行優先度を出力する。

1.2 目的

MIRAI Shield の主な目的は，IIoT インフラにおける従来暗号から PQC への円滑で実用的かつ理解しやすい移行を支援することである。表計算シート形式の入力方式と Purdue レイヤモデリングの手法は，産業分野の実務者にとっての使いやすさを向上させ，移行計画を透明で，構造的で，再現可能なものにすることを意図して設計されている。

2. プラットフォームのワークフロー

MIRAI Shield は，Figure 1 に示すワークフローに従う。このプロセスは，システムモデリングとインベントリ作成から始まり，その情報を Unified PQC- and Migration-Aware Conduit Inventory (UPMI) に統合する。ポリシーエンジンが現実的に展開可能なセキュリティカテゴリを決定した後，プロファイルレコメンダが候補アルゴリズムをランク付けし，信頼度と移行優先度を報告する。

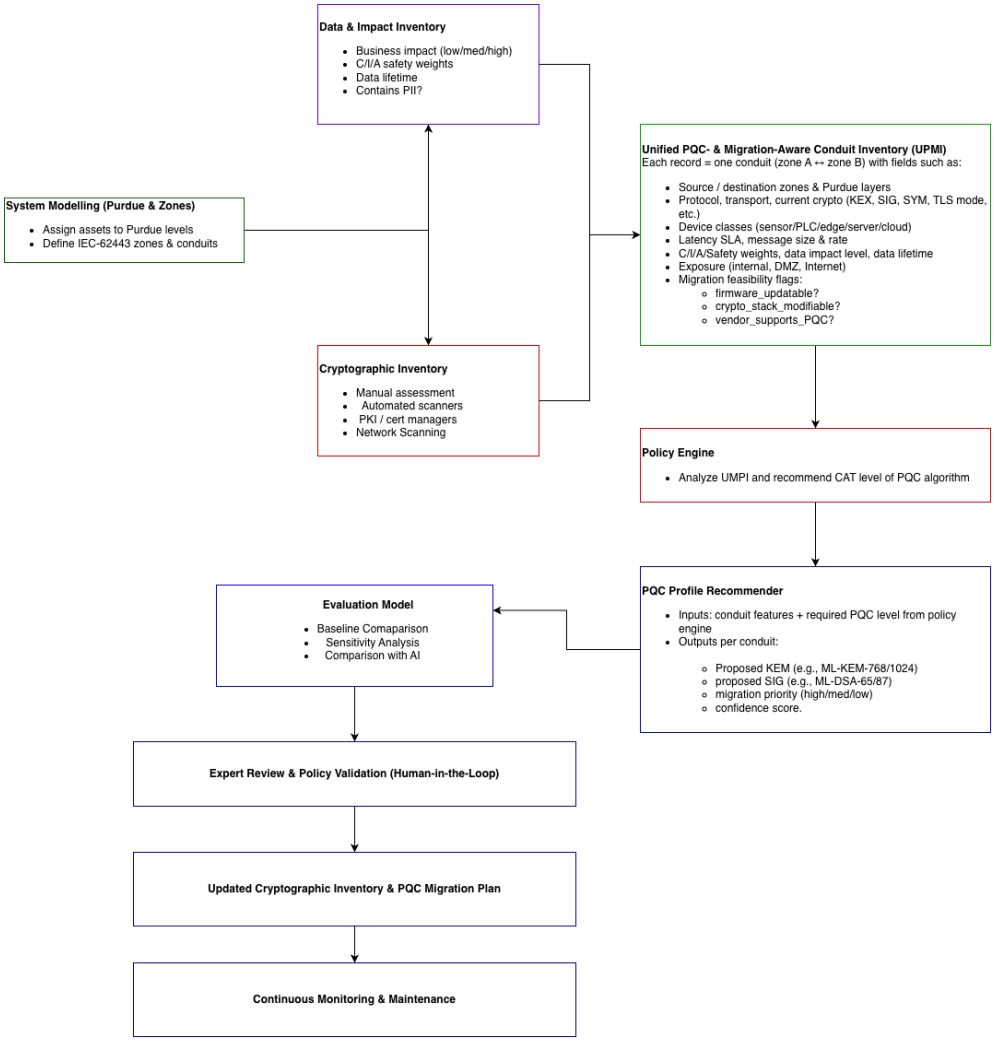

Figure 1: PQC 移行のための MIRAI Shield アーキテクチャ。

2.1 ステップごとの流れ

1. 対象インフラをモデル化し，資産を Purdue レイヤおよびゾーンに割り当てる。
2. **Data and Impact Inventory** と **Cryptographic Inventory** を作成する。
3. これらの入力を **UPMI** データセットに統合する。
4. データセットを MIRAI Shield プラットフォームにアップロードする。
5. **Policy Engine** を実行し，各コンジットに対して展開可能な PQC カテゴリを取得する。
6. **Profile Recommender** を実行し，ランク付けされたアルゴリズム，最適候補，信頼度スコア，および移行優先度を取得する。
7. 出力を確認し，専門家による検証を通じて移行計画を改善する。

3. システムモデリングとインベントリ作成

3.1 Purdue レイヤモデリング

データセットを準備する前に，実務者は Purdue レイヤとセキュリティゾーンを用いて対象システムをモデル化する必要がある。各通信関係は，送信元ゾーンと宛先ゾーンの間のコンジットとして表現されるべきである。これにより，運用コンテキスト，データ感度，および展開可能性を最終的な PQC 推奨に結び付けることが可能になる。

3.2 Data and Impact Inventory

資産を適切な Purdue レイヤに割り当てた後，**Data and Impact Inventory** を作成する必要がある。このインベントリは，各資産またはコンジットの運用上およびセキュリティ上の重要性を記録する。典型的な情報には，データ影響レベル，機密性／完全性／可用性の重み，データ寿命，ビジネス影響，安全性の感度，および個人識別情報の有無が含まれる。

3.3 Cryptographic Inventory

組織全体で現在展開されている暗号システム，アルゴリズム，プロトコル，および保護機構を把握するために，別途 **Cryptographic Inventory** を作成する必要がある。これには，暗号化方式，認証方式，証明書，鍵管理システム，およびプロトコルレベルのセキュリティ制御など，ハードウェアおよびソフトウェアの両方の構成要素が含まれる。このインベントリは，手動評価，自動スキャナ，PKI または証明書管理レビュー，およびネットワークスキャンを通じて収集できる。

4. アップロード用データセットの準備

4.1 重要なフォーマット規則

Web サイトへのアップロードを正しく機能させるために，データセットは次の規則に従う必要がある。

- プラットフォームが期待する正確な列名を使用すること。
- すべての行で値の表記を統一すること。
- Purdue layer，security protocol，device type，exposure，authentication type などの重要な項目を空欄にしないこと。
- 1 行を 1 つのコンジットまたは通信経路として使用すること。
- カテゴリ値は Section 4.3 に示す受け入れ可能な形式で記述すること。

4.2 推奨される必須列

アップロードするデータセットには，次の列を含める必要がある。

`ID`, `Purdue Layer`, `Sec_Protocol`, `Sec_Placement`, `Data Lifetime`, `Device`, `Exposure`, `C_level`, `A_level`, `I_level`, `Business impact`, `Protocols`, `Has_Enc`, `Auth_Type`, `Firmware upgradable`, `Crypto modifiable`, `Vendor Support of PQC`, and `Contains PII`.

`Conduit Name`, `Zones`, `Has_Int`, `DATA Type`, `S_level` などの追加の説明用列も，利用可能であれば含めることができる。

4.3 Unified PQC and Migration aware conduit inventory (UPMI) 属性ガイド

UPMI では，すべてのインベントリ情報とそれらに関連する関係性が，対応する Purdue レイヤのマッピングとともに単一の dataframe に統合され，後続の解析のための 1 つの統一ビューを構成する。

Table 1: UPMI データセット項目と推奨値形式。

Field	Purpose	Recommended values / format
ID	各コンジットエントリの一意識別子。	行ごとに一意の値を使用する。例えば SG-01 や CHEM-12。
Conduit Name	コンジットの人が読める名称。	自由記述。例えば Spot Market - Energy Market Subsystem。
Zones	コンジットで接続される送信元ゾーンと宛先ゾーン。	モデル化されたシステムに基づく自由記述。例えば Market - Energy Market Subsystem。
Purdue Layer	通信経路がまたがるレイヤ範囲。	L0, L1, L2, L3, L4, L5 を含む値を使用する。例えば L2-L2, L3-L2, L5-L4。現在のロジックでは，該当する場合は L2-L2 を正確にそのまま記述する必要がある。
Protocols	コンジットの主なプロトコルおよびリンクコンテキスト。	Ethernet, Wireless, VPN, TLS, WebService, Internet などの認識可能なプロトコルまたはネットワーク用語，あるいは Ethernet/TLS のような組み合わせを含める。
Sec_Protocol	現在コンジットを保護しているセキュリティプロトコル。	TLS, IPsec, MACsec, SSH, DLMS, COSEM, OPC UA, DNP3, BACnet/SC, HTTPS などの認識されるセキュリティプロトコルを含める。例：TLS 1.2/1.3。
Has_Enc	暗号化が存在するかどうか。	YES または NO を使用する。Partial も文書化のために残せるが，プラットフォーム実行上は YES/NO が最も安全な形式である。
Auth_Type	コンジットで使用される認証方式。	現在のレコメンダロジックでは ServerCert または MutCert を使用する。PSK や None などの他の説明的な値は文書化のために保存できるが，プラットフォームは証明書ベースの形式を最も確実に認識する。
Has_Int	完全性保護が存在するかどうか。	推奨値：YES, NO, Partial。
Sec_Placement	セキュリティが適用される場所。	推奨値：EndToEnd, GatewayOnly, LAN only, WAN, local only。GatewayOnly は，制約のある展開で特に重要である。
DATA Type	コンジット上で扱われる主なデータ種別。	telemetry, control, logs, commands などの自由記述。
C_level	機密性レベル。	H, M, L を使用する。
A_level	可用性レベル。	H, M, L を使用する。
I_level	完全性レベル。	H, M, L を使用する。
S_level	安全性感度レベル。	含める場合は H, M, L を使用する。
Business impact	コンジットが侵害された場合の運用上または事業上の影響。	アップロード互換性のためには L/M/H を推奨する。別の場所で説明的な尺度を用いる場合は，一貫性を保つこと。

Field	Purpose	Recommended values / format
Data Lifetime	必要な保護寿命。	推奨値：10y+, 5-10y, 1-5y, <1y。10y または 10y+ は，最も高い長期要件として解釈される。
Contains PII	個人識別情報が含まれるかどうか。	Yes または No を使用する。
Exposure	信頼されていないネットワークまたは外部ネットワークへの露出。	Low, Internet, WAN, LAN/Internal, Isolated など，一貫した値を使用する。説明ラベルが必要な場合は，すべての記述の末尾に high, low, medium のような程度を付ける必要がある。例えば Internet / external B2B (High)。
Device	コンジットに関与するデバイスクラス。	説明的なデバイス名を用い，クラスを明確に示す。例えば Substation controller (Resource Constrained), HMI server (Servers), Gateway。Resource Constrained, Servers, gateway は認識上重要なキーワードである。
Firmware upgradable	ファームウェアを更新可能かどうか。	Yes, No, Unknown を使用する。
Crypto modifiable	暗号スタックを変更または置換可能かどうか。	Yes, No, Unknown を使用する。
Vendor Support of PQC	現時点でのベンダによる PQC サポート。	Yes, No, Unknown を使用する。

5. プラットフォームの使用方法

5.1 ステップ 1: データセットをアップロードする

UPMI データセットを CSV または表計算シート形式で準備し，MIRAI Shield インタフェースからアップロードする。各行は 1 つのコンジットを表す必要がある。アップロード後，プラットフォームは列を解析し，ポリシーエンジン用に準備する。

5.2 ステップ 2: Policy Engine を実行する

データセットのアップロード完了後に **Policy Engine** をクリックする。ポリシーエンジンは，必要な保護レベルと，対象通信経路上でその展開が現実的に可能かどうかを判断するために，ルールベースの評価を行う。

主なポリシー出力

主要な出力は **Possible Deployed CAT** である。これは，セキュリティ要件と実運用上の制約のバランスをとったうえで，当該コンジットに対して現実的に展開可能な PQC セキュリティカテゴリを示す。

典型的な出力は次のとおりである。

- **CAT-1**
- **CAT-3**
- **CAT-5**

- 現時点で安全な展開が実行不可能な場合の **PQC unprotected**

5.3 ステップ 3: Profile Recommender を実行する

ポリシー結果が得られた後に **Profile Recommender** をクリックする。レコメンダは，展開カテゴリ，ベンチマークデータ，デバイスクラス，プロトコルコンテキスト，および遅延に関する前提を組み合わせて，候補となる KEM および署名アルゴリズムをランク付けする。

遅延とデバイスの解釈

現在の実装では，Purdue レイヤのコンテキストを次のように遅延バケットおよびデバイス制約へ対応付けている。

- **Strict bucket**: `L0`, `L1`, `L2` を含むレイヤ
- **Medium bucket**: `L3` を含むレイヤ
- **Loose bucket**: `L4` または `L5` を含むレイヤ

また，レコメンダは constrained，embedded，server-like のデバイスクラスを区別しており，これが CPU コスト，バイトオーバーヘッド，および RAM 使用量の重みに影響する。

レコメンダの出力

各コンジットについて，インタフェースは次を報告する。

- **Algorithm Rank**
- **Best Algorithm**
- **Confidence Score**
- **Migration Priority**

6. 出力の解釈

6.1 Algorithm rank

ランク付けされた一覧は，選択されたデバイス条件および遅延条件の下で，候補アルゴリズムを最も適したものから最も適さないものへ順に並べる。ランキングは utility に基づいているため，utility 値が低いほど全体適合性が高いことを示す。

6.2 Best algorithm

Best algorithm は，選択されたコンジットおよびセキュリティカテゴリに対して実現可能な選択肢の中で最上位に位置付けられた候補である。

6.3 Confidence score

Confidence score は，システムが上位候補を近接する代替候補よりどの程度強く支持しているかを示す。これは utility ランキングから導出されるものであり，直接的な暗号学的安全性保証ではなく，意思決定支援のためのスコアとして解釈すべきである。

6.4 Migration priority

Migration priority は，リスクの緊急性，実現可能性，および移行時の摩擦を組み合わせたものである。現在の設計では，優先度スコアはリスク緊急度，実現可能性，およびコスト／摩擦の重み付き組合せから計算され，その後 **High**, **Medium**, **Low** に対応付けられる。実務上は，高優先

度のコンジットとは，保護の重要性が高く，かつ移行計画の早い段階で対応することが現実的なコンジットを意味する。

7. 推奨レビュー手順

MIRAI Shield を効果的に利用するために，実務者は次の順序で出力を確認することが望ましい。

1. アップロードしたデータセットが正しいスキーマと正規化された値を使用していることを確認する。
2. ポリシーエンジンが返した **Possible Deployed CAT** を確認する。
3. **Best Algorithm** と上位数個の代替候補を確認する。
4. **Confidence Score** を確認する。信頼度が低い場合は，候補間の競合が接近していることを示す。
5. **Migration Priority** を用いて，コンジットごとの段階的な展開計画を立てる。
6. 運用展開の前に，専門家レビューを通じて最終判断を検証する。

8. よくあるアップロード時の誤り

- 必須ヘッダの列名を変更したり，大文字小文字を変えてしまうこと。
- 行ごとにカテゴリラベルの表記が一貫していないこと。
- プラットフォームが `ServerCert` または `MutCert` を期待しているにもかかわらず，自由形式の認証ラベルを使うこと。
- デバイスクラスを明確に `Resource Constrained`, `Servers`, `gateway` と記さず，曖昧なままにすること。
- intended level を明確に示さないまま，露出に関する表現を混在させること。
- ゲートウェイベースの展開を意図している constrained case において，`GatewayOnly` を省略すること。

8.1 デフォルト重みとユーザによるカスタマイズ

本節では，MIRAI Shield で用いられるデフォルトのスコアリング式と，組織ごとの要件に応じて重みを変更できる考え方について説明する．本プラットフォームは，データセットのアップロードおよびポリシーエンジンの実行後，追加設定なしでもデフォルト値によってそのまま利用できるよう設計されている．一方で，運用要件，規制要件，あるいは事業上の方針に応じて，ユーザは最終的なバックエンド計算を実行する前に，プラットフォーム上で重みを変更することも可能である．

プロファイル順位付けに用いるユーティリティスコア　各候補プロファイル p に対して，MIRAI Shield は正規化済みのユーティリティスコアを計算する．ユーティリティ値が小さいほど，対象の通信路に対してより適した候補であることを意味する．

一般形は次式で与えられる．

$$U_i(p) = w_1\, z_{\mathrm{CPU}}(x_i, p) + w_2\, z_{\mathrm{BYTES}}(x_i, p) + w_3\, z_{\mathrm{RAM}}(x_i, p) + w_4\, \mathrm{RiskPenalty}(p) + w_5\, \mathrm{InteropPenalty}(p). \tag{1}$$

ここで，正規化項は以下のように定義される．

$$z_{\mathrm{CPU}}(p) = \frac{\mathrm{CPU}(p)}{\mathrm{CPU}_{\mathrm{ref}}(\mathrm{device_class})}, \qquad z_{\mathrm{RAM}}(p) = \frac{\mathrm{RAM}(p)}{\mathrm{RAM}_{\mathrm{ref}}(\mathrm{device_class})}, \qquad z_{\mathrm{BYTES}}(p) = \frac{\mathrm{BYTES}(p)}{\mathrm{BYTES}_{\mathrm{ref}}}. \tag{2}$$

現行版のフレームワークでは，RiskPenalty(p) と InteropPenalty(p) は実装上のスコア計算には含めていない．したがって，実際に用いられるユーティリティ式は，CPU，通信バイト量，および必要に応じて RAM のみに基づく形となる．

デフォルトのユーティリティ重み　MIRAI Shield では，サーバクラス機器と MCU ／組込み機器とで現実的な制約が異なるため，デフォルト重みを分けている．

サーバまたはゲートウェイ環境では，デフォルトのユーティリティスコアは次式である．

$$U_{\mathrm{server}}(p) = 0.7\, z_{\mathrm{CPU}}(p) + 0.3\, z_{\mathrm{BYTES}}(p). \tag{3}$$

MCU または組込み環境では，デフォルトのユーティリティスコアは次式である．

$$U_{\mathrm{mcu}}(p) = 0.6\, z_{\mathrm{CPU}}(p) + 0.2\, z_{\mathrm{BYTES}}(p) + 0.2\, z_{\mathrm{RAM}}(p). \tag{4}$$

これらのデフォルト値は，実運用における制約の違いを反映している．サーバやゲートウェイでは，PQC 処理時における RAM の圧迫は，計算時間や通信オーバーヘッドに比べて通常は支配的ではない．そのため，サーバ側のデフォルト式では RAM の重みを 0 としている．一方で，MCU や組込み機器ではメモリ制約が厳しいことが多いため，RAM を明示的にスコアに含めている．

順位付け規則　候補プロファイルは，ユーティリティ値の昇順で順位付けされる．

$$\mathrm{ranked_profiles}_i = \mathrm{sort}_{p \in P_i}\, U_i(p) \quad \text{(ascending)}. \tag{5}$$

認証方式として相互証明書認証が選択される場合には，送信側および受信側の双方のコストを考慮してスコアを計算する．

信頼度スコア　順位付け後，MIRAI Shield はソフトマックス型の信頼度スコアを計算し，最上位候補が他の候補に比べてどの程度優位であるかを示す．まず，

$$U_{\min} = \min_j U_j \tag{6}$$

とし，温度パラメータを

$$\tau = \alpha\, U_{\min} \tag{7}$$

で定義する．ここで，α は厳しさを調整するパラメータであり，デフォルト値は次の通りである．

$$\alpha = 0.05. \tag{8}$$

ユーティリティ値は小さいほど望ましいため，シフト後スコアは次式で与えられる．

$$S_i = -\frac{U_i - U_{\min}}{\tau}, \tag{9}$$

続いて，

$$e_i = e^{S_i}, \qquad P_i = \frac{e_i}{\sum_j e_j}. \tag{10}$$

信頼度スコアが高いほど，最上位候補が他の実行可能候補よりも明確に優れていることを意味する．

移行優先度スコア　MIRAI Shield は，各通信路に対して，リスク緊急度，実現可能性，およびコスト／摩擦を組み合わせた移行優先度スコアも計算する．

リスク緊急度スコアは次式である．

$$R_i = 0.45E_i + 0.35I_i + 0.20L_i, \tag{11}$$

ここで，E_i は露出度スコア，I_i は業務影響スコア，L_i はデータ保存期間スコアを表す．

実現可能性スコアは次式である．

$$F_i = 0.40FW_i + 0.40CM_i + 0.20V_i, \tag{12}$$

ここで，FW_i はファームウェア更新可能性，CM_i は暗号変更可能性，V_i はベンダによる PQC 対応状況を表す．

コスト／摩擦スコアは次式である．

$$C_i = 0.40S_i + 0.30Pu_i + 0.30D_i, \tag{13}$$

ここで，S_i はセキュリティ配置に伴う摩擦，Pu_i は Purdue レイヤに基づく摩擦，D_i はデバイス種別に基づく摩擦を表す．

最終的な移行優先度スコアは次式で与えられる．

$$\text{PriorityScore}_i = 0.50R_i + 0.30F_i - 0.20C_i. \tag{14}$$

優先度レベルは次のように解釈される．

$$\text{PriorityLevel}_i = \begin{cases} \text{High}, & \text{PriorityScore}_i \geq 0.50, \\ \text{Medium}, & 0.30 \leq \text{PriorityScore}_i < 0.50, \\ \text{Low}, & \text{PriorityScore}_i < 0.30. \end{cases} \tag{15}$$

これらのデフォルト値を採用する理由　デフォルト係数は，IIoT における実践的な移行計画に向けたバランスの取れた基準設定として定められている．ユーティリティスコアでは，サーバクラス環境では CPU と通信オーバーヘッドを重視し，制約の厳しい環境では RAM も考慮する．また，移行優先度スコアでは，まずリスク緊急度を重視し，次に実現可能性を評価し，コスト／摩擦は優先度を下げる要素として扱う．この設計により，本フレームワークは，安全性だけでなく実際の導入可能性も踏まえた推薦を行うことができる．

カスタマイズが適切な場合　組織固有のポリシーが特に存在しない場合には，デフォルト重みをそのまま使用することが推奨される．一方で，運用上の優先事項が基準想定と異なる場合には，重みの変更が適切となることがある．例えば，非常に制約の厳しい端末に対して RAM の重みを高める場合，安全性が重要な環境において露出度や業務影響の重みを高める場合，あるいは迅速な移行を戦略的に優先するためにコストペナルティを下げる場合などが考えられる．

プラットフォーム上での動作 MIRAI Shield は，デフォルトでは本節で示した事前定義済みの重みに基づいてすべての計算を実行する．ただし，ユーザがプラットフォーム上の入力設定でカスタム重みを指定した場合には，その値がバックエンドへ送信され，同一の数式に適用される．すなわち，変更されるのは係数のみであり，計算式自体は変わらない．この設計により，組織ごとに調整を行った場合でも，スコアリングの論理は一貫性と説明可能性を維持できる．

推奨事項 カスタム重みを用いる場合には，その設定値を組織の移行ポリシーの一部として文書化することが望ましい．これにより，得られる推薦結果の透明性，再現性，およびレビュー可能性を確保できる．

9. 説明用ケーススタディ: Smart Grid UPMI 例

MIRAI Shield の実用的な使用方法を示すために，構造化された **UPMI** データセットを用いた説明用の **Smart Grid ケーススタディ** を本マニュアルの最後に示すことができる。この例は，実際の産業データセットがどのように統合インベントリへ変換され，その後 **Policy Engine** と **PQC Profile Recommender** によって処理されるかを，実務者が理解する助けとなる。

このケーススタディでは，データセットの各行が 1 つの **communication conduit** を表す。データセットには，コンジット識別子，コンジット名，ゾーン，Purdue layer，protocols，現在の security protocol，security placement，data type，C/A/I/S levels，business impact，data lifetime，exposure level，device type，および migration-feasibility attributes など，MIRAI Shield が必要とする主要属性が含まれている。これらの属性を 1 つの統一構造に整理することで，データセットはフレームワーク内で直接解析可能になる。

Smart Grid の例が特に有用なのは，enterprise-level や market-facing communications から，operational，substation，field-level communications に至るまで，複数の **Purdue layers** にまたがるコンジットレコードを含んでいるためである。その結果，このケーススタディは，通信コンテキストの違いがどのように異なるポリシー結果，必要な PQC セキュリティカテゴリ，および移行優先度につながるかを示している。

この例はまた，MIRAI Shield がさまざまなセキュリティ条件をどのように扱えるかも示している。あるコンジットはすでにプロトコルレベルの保護を使用している一方で，別のコンジットはゲートウェイベースの保護，部分的な保護，あるいは暗号保護を全く備えていない場合もある。このように多様なレコードを処理することにより，フレームワークはポリシーエンジンがどのように *possible deployed CAT level* を決定し，レコメンダがどのように **confidence score**, **best algorithm**, **migration priority** とともに候補アルゴリズムのランク付き一覧を生成するかを示すことができる。

したがって，この Smart Grid ケーススタディは，独自のデータセットの準備方法およびフレームワークの最終出力の解釈方法を理解したい利用者にとって，実用的な参照例となる。また，MIRAI Shield が実際の IIoT 環境において，構造化され，説明可能で，移行を意識した PQC 計画をどのように支援できるかを示す明確な例でもある。

9.1 Example Dataset Link

デモ用に使用した Smart Grid UPMI の例は，次のスプレッドシートリンクで提供できる。

`https://docs.google.com/spreadsheets/d/1V5GDf1WEGMkfJR2hT2p9wkd1PuCQxXgk66CxyIcPrKM/edit?usp`

10. 結論

MIRAI Shield は，産業システムモデリングから移行志向の PQC 推奨に至るまでの実用的な道筋を提供する。Purdue レイヤを考慮したインベントリ，ルールベースのポリシーエンジン，および utility ベースのプロファイルレコメンダを組み合わせることにより，本プラットフォームは，生のインフラデータを保護レベル，展開可能なアルゴリズム，および移行順序に関する構造化さ

れた意思決定へと変換することを実務者に可能にする。このフレームワークに関する完全なアルゴリズム詳細は，full algorithm specification guide section で参照可能である。